04-02-linear-regression-homework

January 26, 2026

```
<div style='float:left;'><h1>CPSC 4300/6300: Applied Data Science</h1></div>
<img style="float: right; padding-right: 10px; width: 65px" src="https://bsethwalker.github.io/
```

1 Week 4 | Homework : Linear Regression

Clemson University Instructor(s): Tim Ransom

1.1 Learning goals

- Define simple linear regression and its assumptions.
- Calculate the coefficients of a simple linear regression model.
- Interpret the R-squared value for a linear regression model.
- Construct a polynomial regression model using Python.
- Evaluate the performance of a multiple linear regression model.

1.2 INSTRUCTIONS

- To submit your assignment follow the instructions given in Canvas.
- Restart the kernel and run the whole notebook again before you submit.
- As much as possible, try and stick to the hints and functions we import at the top of the homework, as those are the ideas and tools the class supports and is aiming to teach. And if a problem specifies a particular library you're required to use that library, and possibly others from the import list.
- Please use .head() when viewing data. Do not submit a notebook that is excessively long because output was not suppressed or otherwise limited.

1.3 Overview

You are hired by the administrators of the Capital Bikeshare program program in Washington D.C., to **help them predict the hourly demand for rental bikes** and **give them suggestions on how to increase their revenue**. Your task is to prepare a short report summarizing your findings and make recommendations.

The predicted hourly demand could be used for planning the number of bikes that need to be available in the system at any given hour of the day. It costs the program money if bike stations are full and bikes cannot be returned, or empty and there are no bikes available. You will use multiple linear regression and polynomial regression and will explore ridge and lasso regression to

predict bike usage. The goal is to build a regression model that can predict the total number of bike rentals in a given hour of the day, based on all available information given to you.

An example of a suggestion to increase revenue might be to offer discounts during certain times of the day either during holidays or non-holidays. Your suggestions will depend on your observations of the seasonality of ridership.

The data for this problem were collected from the Capital Bikeshare program over the course of two years (2011 and 2012).

1.4 About

For this regression homework you are provided with initial dataset in the file `data/BSS_hour_raw.csv`. You will first add features that will help with the analysis and then separate the data into training and test sets. Each row in this file represents the number of rides by registered users and casual users in a given hour of a specific date. There are 12 attributes in total describing besides the number of users the weather if it is a holiday or not etc:

- `dteday` (date in the format YYYY-MM-DD, e.g. 2011-01-01)
- `season` (1 = winter, 2 = spring, 3 = summer, 4 = fall)
- `hour` (0 for 12 midnight, 1 for 1:00am, 23 for 11:00pm)
- `weekday` (0 through 6, with 0 denoting Sunday)
- `holiday` (1 = the day is a holiday, 0 = otherwise)
- `weather`
  - 1: Clear, Few clouds, Partly cloudy, Partly cloudy
  - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
  - 3: Light Snow, Light Rain + Thunderstorm
  - 4: Heavy Rain + Thunderstorm + Mist, Snow + Fog
- `temp` (temperature in Celsius, normalized)
- `atemp` (apparent temperature, or relative outdoor temperature, in Celsius, normalized)
- `hum` (relative humidity, normalized)
- `windspeed` (wind speed, normalized)
- `casual` (number of rides that day made by casual riders, not registered in the system)
- `registered` (number of rides that day made by registered riders)

---

[1]:
```
""" RUN THIS CELL TO GET THE RIGHT FORMATTING """
import requests
from IPython.core.display import HTML
css_file = 'https://raw.githubusercontent.com/bsethwalker/clemson-cs4300/main/
 ↪css/cpsc6300.css'
styles = requests.get(css_file).text
HTML(styles)
```

[1]:
```
<IPython.core.display.HTML object>
```

1.5 Use only the libraries below:

[2]:
```
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt

import statsmodels.api as sm
from statsmodels.api import OLS

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import RidgeCV
from sklearn.linear_model import LassoCV
from sklearn.metrics import r2_score
from pandas.plotting import scatter_matrix
import matplotcheck.notebook as nb
from matplotcheck.base import PlotTester
from matplotlib.patches import PathPatch
```

Part 1 - Data Processing

In this section, we read in the data and begin one of the most important analytic steps: verifying that the data is what it claims to be.

Exerise 1.1

- Load the dataset from the csv file `data/BSS_hour_raw.csv` into a pandas dataframe named `bikes_df_raw`.
- Check basic statistics and data types to inspect variable ranges and averages
- Do any of the variables ranges or averages seem suspect?
- Do the data types make sense?

[5]:
```
"""Write your code for exercise-1.1 here:"""

# your code here
bikes_df_raw = pd.read_csv('data/BSS_hour_raw.csv')
```

[ ]:
```

```

It's good practice to have some fast action smell checks for the data - ask yourself these questions and check for the charactetristics listed under them when interpreting data to find early signs that somethings gone wrong.

1. Do Any of the Variables' Ranges or Averages Seem Suspect?

From the summary statistics, none of the variables appear to have suspicious ranges or averages: - All categorical variables (season, holiday, workingday, weather) have reasonable and consistent

ranges. - The numeric variables (temp, atemp, hum, windspeed, casual, registered) also have expected ranges, with no negative values or unexpected spikes. - The normalization between 0 and 1 for variables like temperature, atemp, humidity, and windspeed is appropriate and makes sense for preprocessed data. - The data doesn't show any major outliers or values that seem out of place given the context of a bike-sharing dataset.

2. Do the Data Types Make Sense?

The data types also appear to make sense for each variable: - dteday is an object (likely a string representing a date). It might be more convenient if it were converted to a datetime type for better manipulation in analysis. - Categorical variables (season, holiday, workingday, weather, etc.) are stored as integers, which is expected. - Numerical variables (temp, atemp, hum, windspeed, casual, registered) are stored as either int64 or float64, which are appropriate types.

Next steps following above statistical analysis: Convert dteday from an object type to a datetime type for easier manipulation.

Exercise 1.2

- Notice that the variable in column `dteday` is a pandas `object`, which is **not** useful when you want to extract the elements of the date such as the year, month, and day.
- Convert `dteday` into a `datetime` object to prepare it for later analysis.

Hint: Refer to this page pandas.to_datetime

[9]:
```
"""Write your code for exercise-1.2 here:"""

# your code here
bikes_df_raw['dteday'] = pd.to_datetime(bikes_df_raw['dteday'])
```

[ ]:
```

```

Exercise 1.3

Create three new columns in the dataframe:

- `year` with 0 for 2011, 1 for 2012, etc.
- `month` with 1 through 12, with 1 denoting January.
- `counts` with the total number of bike rentals(sum of casual and registered bike rentals) for that **hour** (this is the response variable for later).

[59]:
```
"""Write your code for exercise-1.3 here:"""

# your code here
bikes_df_raw['year'] = bikes_df_raw['dteday'].dt.year - bikes_df_raw['dteday'].
 ↪dt.year.min()
bikes_df_raw['month'] = bikes_df_raw['dteday'].dt.month
bikes_df_raw['counts'] = bikes_df_raw['casual'] + bikes_df_raw['registered']
```

[ ]:
```

```

Part 2- Exploratory Data Analysis

In this section we begin hunting for patterns in ridership that shed light on who uses the service and why.

Exercise 2.1

Create a new dataframe named **`bikes_by_day`** with the following subset of attributes from the previous dataset and with each entry being just **one** day:

- `dteday`, the timestamp for that day (fine to set to noon or any other time)
- `weekday`, the day of the week
- `weather`, the most severe weather that day
- `season`, the season that day falls in
- `temp`, the average temperature
- `atemp`, the average atemp that day
- `windspeed`, the average windspeed that day
- `hum`, the average humidity that day
- `casual`, the **total** number of rentals by casual users
- `registered`, the **total** number of rentals by registered users
- `counts`, the **total** number of rentals of that day

Make a plot showing the *distribution* of the number of casual and registered riders on each day of the week.

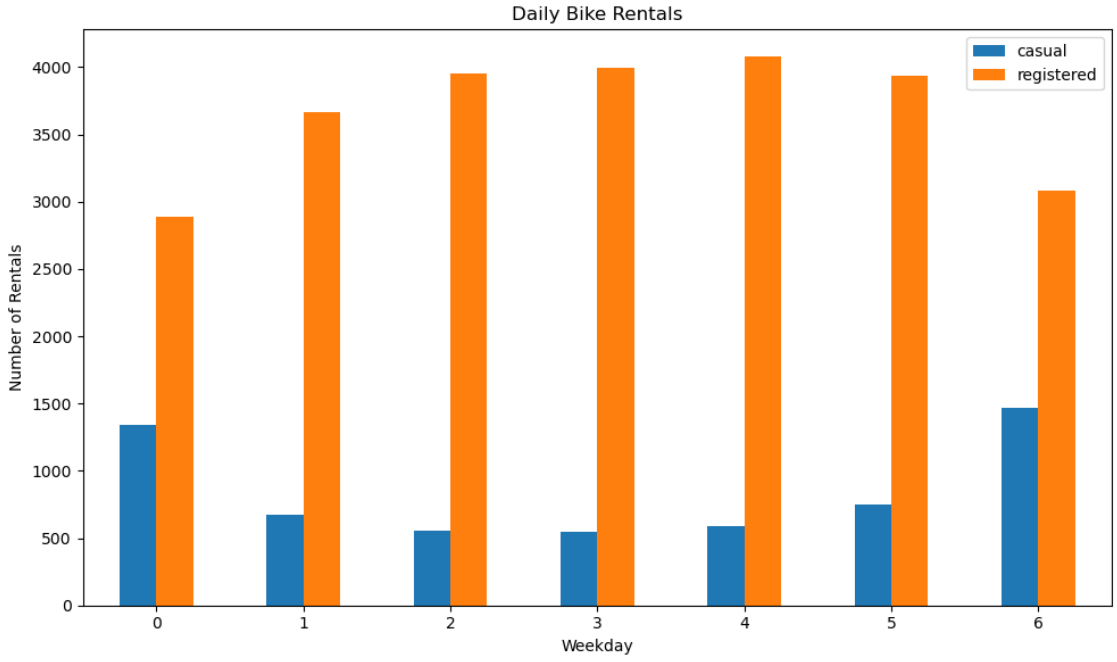
1. Create the bar plot, you need to set up the Figure and Axes objects using `plt.subplots()`.
   - These two objects (fig and ax) will allow you to control various properties of the figure and plot.
   - `fig` is the Figure object: It serves as the overall container for the plot.
   - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.

     `Example code: fig, ax = plt.subplots(figsize=(10, 6))`
2. Customize the Plot:
   - You need to set the title of the plot, the labels for the x-axis and y-axis, and format the ticks on the x-axis for better readability.

Hint: - Helpful to use panda's `.groupby()` command. - Refer to this documentation pandas.DataFrame.groupby for more information.

[69]:
```
"""Write your code for exercise-2.1 here:"""

# your code here
bikes_by_day = bikes_df_raw.groupby("dteday", as_index=False).agg({
    "weekday": "first",
    "weather": "max",
    "season": "first",
    "temp": "mean",
    "atemp": "mean",
    "windspeed": "mean",
    "hum": "mean",
    "casual": "sum",
```

```
    "registered": "sum",
    "counts": "sum",

})

weekday_counts =(
    bikes_by_day
    .groupby('weekday')[['casual', 'registered']]
    .mean()
)

fig, ax = plt.subplots(figsize=(10, 6))

weekday_counts.plot(kind='bar', ax=ax)

ax.set_title('Daily Bike Rentals')
ax.set_xlabel('Weekday')
ax.set_ylabel('Number of Rentals')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)

plt.tight_layout()
plt.show()
```



[ ]:

Exercise 2.2

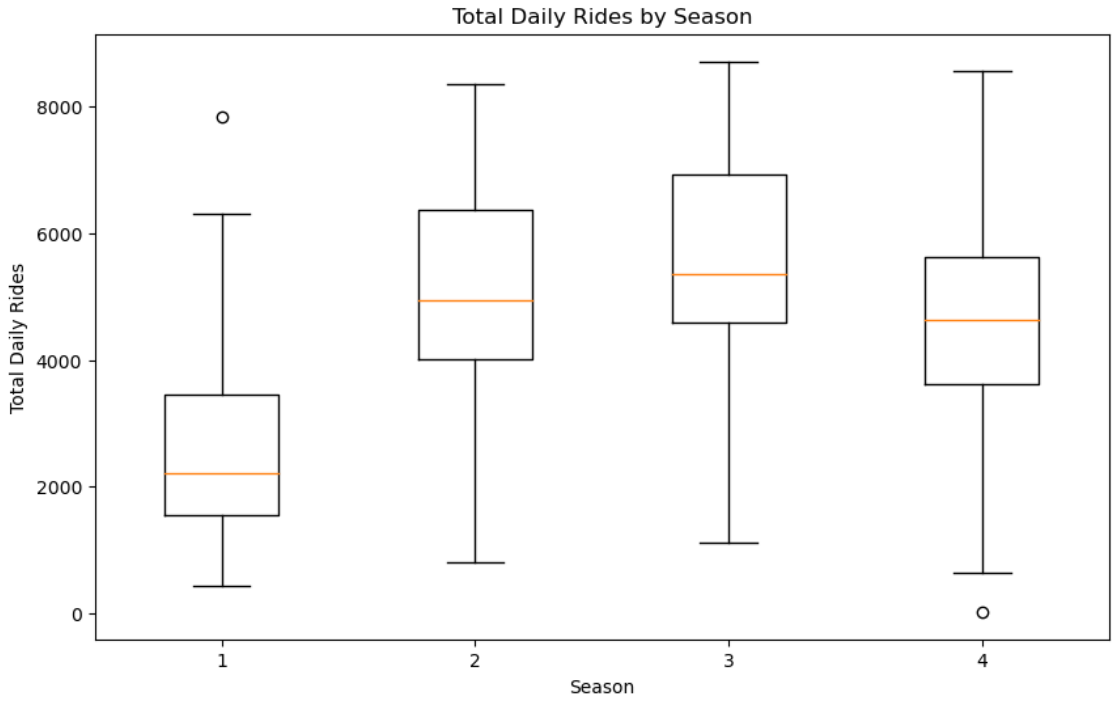
- Use `bikes_by_day` to visualize how the distribution of **total number of rides** per day (casual and registered riders combined) varies with the **season**.

    1. Create the box plot, you need to set up the Figure and Axes objects using `plt.subplots()`.
        - These two objects (fig and ax) will allow you to control various properties of the figure and plot.
        - `fig` is the Figure object: It serves as the overall container for the plot.
        - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.
          `Example code: fig, ax = plt.subplots(figsize=(10, 6))`
    2. Customize the Plot:
        - You need to set the title of the plot, the labels for the x-axis and y-axis, and format the ticks on the x-axis for better readability.

Investigating outliers 1. Here we use the pyplot's boxplot function definition of an outlier as any value 1.5 times the IQR above the 75th percentile or 1.5 times the IQR below the 25th percentiles. 2. Store the outliers in dataframe called `outliers`. 3. If you see any outliers, identify those dates and investigate if they are a chance occurence, an error in the data collection, or a significant event (an online search of those date(s) might help).

[61]:
```
"""Write your code for exercise-2.2 here:"""

# your code here

seasons = sorted(bikes_by_day['season'].unique())
data = [out for out in (bikes_by_day.loc[bikes_by_day['season'] == s, 'counts'].
 ↪dropna().values for s in seasons)]

fig, ax = plt.subplots(figsize=(10, 6))
ax.boxplot(data, labels=seasons)

ax.set_title('Total Daily Rides by Season')
ax.set_xlabel('Season')
ax.set_ylabel('Total Daily Rides')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)

plt.show

outlier_frames = []
for s in seasons:
    vals = bikes_by_day.loc[bikes_by_day["season"] == s, "counts"].dropna()
    q1 = vals.quantile(0.25)
    q3 = vals.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
```

```
    season_outliers = bikes_by_day.loc[
        (bikes_by_day["season"] == s) & ((bikes_by_day["counts"] < lower) |␣
↪(bikes_by_day["counts"] > upper)),
        ["dteday", "season", "counts"]
    ]
    outlier_frames.append(season_outliers)

outliers = pd.concat(outlier_frames, ignore_index=True)
```



[ ]:

Write the number of outliers you see into a variable called `answer` in the cell below.

[62]:
```
# your code here
answer = outliers.shape[0]
answer
```

[62]: 2

[ ]:

Exercise 2.3

- Convert the categorical attributes (`season`, `month`, `weekday`, `weather`) into multiple binary

attributes using **one-hot encoding** and call this new dataframe `bikes_df`.

[73]:
```
"""Write your code for Exercise-2.3 here:"""


bikes_df = bikes_by_day.copy()


bikes_df["month"] = pd.to_datetime(bikes_df["dteday"]).dt.month

month_for_stratify = bikes_df["month"].copy()


bikes_df = pd.get_dummies(bikes_df, columns=["season", "month", "weekday",␣
 ↪"weather"])
```

[ ]:
```

```

Exercise 2.4

- Split the updated `bikes_df` dataset into a 50-50 train-test split (call them `bikes_train` and `bikes_test`, respectively).
- Do this in a 'stratified' fashion, ensuring that all months are equally represented in each set.
- Use `random_state = 42`, a test set size of `.5`, and stratify on month.

[74]:
```
"""Write your code for Exercise-2.4 here:"""

# your code here

bikes_train, bikes_test = train_test_split(
    bikes_df,
    test_size=0.5,
    random_state=42,
    stratify=month_for_stratify
)
```

[ ]:
```

```

Exercise 2.5

- Although we asked you to create your train and test set, for consistency and easy checking, we ask that for the rest of this problem set you use the train and test set provided in the files `data/BSS_train.csv` and `data/BSS_test.csv`.
- Read these two files into dataframes `BSS_train` and `BSS_test`, respectively.
- Remove the `dteday` column from both the train and the test dataset (its format cannot be used for analysis).

[32]:
```
"""Write your code for Exercise-2.5 here:"""
```

```
# your code here

BSS_train = pd.read_csv("data/BSS_train.csv")
BSS_test = pd.read_csv("data/BSS_test.csv")


BSS_train = BSS_train.drop(columns=["dteday"])
BSS_test = BSS_test.drop(columns=["dteday"])
```

[ ]:

Exercise 2.6

- Use pandas' `scatter_matrix` command to visualize the inter-dependencies among the list of predictors listed below in the training dataset.
  1. Select specific columns from BSS_train as listed in `cor_columns` list in next cell and store this to new dataframe named `sampled_data`.
  2. Randomly select 10% of the rows to make the scatter matrix more manageable.
  3. Create the scatter matrix plot using `sampled_data` dataFrame.
     - In order to make a plot for this exercise, you will have to set up the Figure and Axes objects namely `fig` and `ax` using `plt.subplots()`.
     - These two objects (fig and ax) will allow you to control various properties of the figure and plot.
     - `fig` is the Figure object: It serves as the overall container for the plot.
     - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.
       Example code: `fig, ax = plt.subplots(figsize=(10, 6))`
- Note and comment on any strongly related variables.

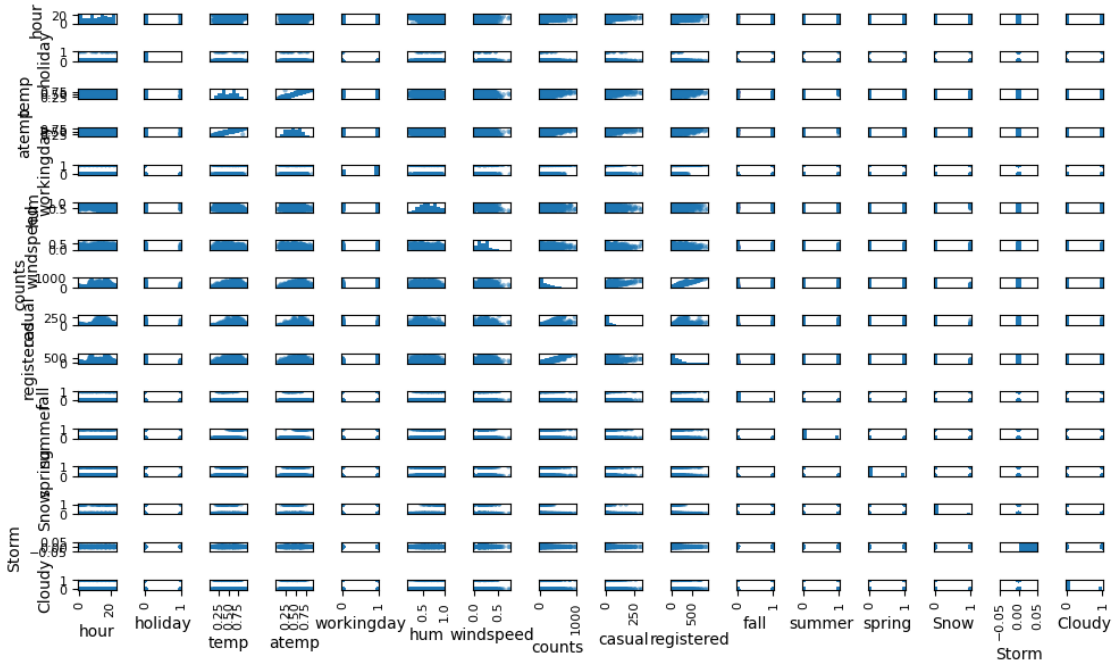
Note:

- **This may take a few minutes to run. You may wish to comment it out until your final submission, or only plot a randomly-selected 10% of the rows.**
- **Refer to this document pandas.DataFrame.sample on how to randomly select 10% of rows from a given dataset.**

[33]:
```
# List of columns to visualize the inter-dependencies among the list of␣
 ↪predictors
cor_columns = [
    'hour', 'holiday', 'temp', 'atemp', 'workingday', 'hum', 'windspeed',
    'counts', 'casual', 'registered', 'fall', 'summer', 'spring',
    'Snow', 'Storm', 'Cloudy'
]
```

[34]:
```
"""Write your code for Exercise-2.6 here:"""

# your code here
```

```
sampled_data = BSS_train[cor_columns]


sampled_data = sampled_data.sample(frac=0.1, random_state=42)


fig, ax = plt.subplots(figsize=(10, 6))


scatter_matrix(sampled_data, ax=ax, figsize=(10, 6), diagonal="hist")

plt.tight_layout()
plt.show()
```



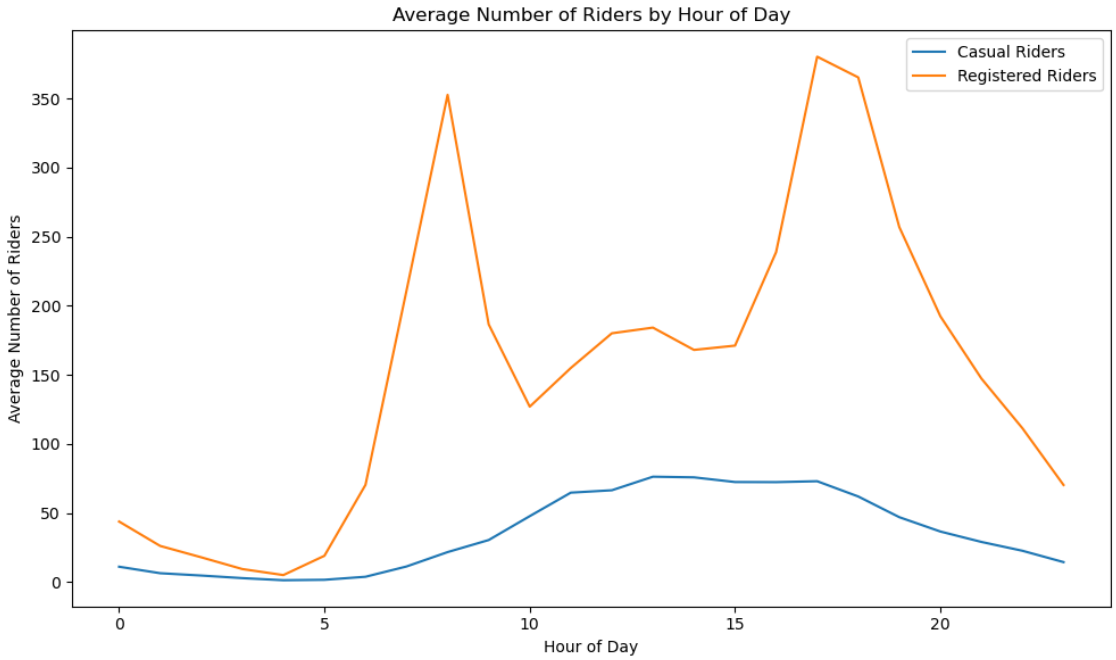
[ ]:

Exercise 2.7

- Make a plot showing the *average* number of casual and registered riders during each hour of the day.
  - In order to make a line plot for this exercise, you will have to set up the Figure and Axes objects namely `fig` and `ax` using `plt.subplots()`. - These two objects (fig and ax) will allow you to control various properties of the figure and plot. - `fig` is the Figure object: It serves as the overall container for the plot. - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc. `python Example code: fig, ax = plt.subplots(figsize=(10, 6))`

- Use `.groupby` and `.aggregate` in order to calculate the average number of casual and registered riders.
- Comment on the trends you observe.

[35]:
```
"""Write your code for Exercise-2.7 here:"""

# your code here

hourly_avg = (
    BSS_train
    .groupby("hour")
    .aggregate({
        "casual": "mean",
        "registered": "mean"
    })
)


fig, ax = plt.subplots(figsize=(10, 6))


ax.plot(hourly_avg.index, hourly_avg["casual"], label="Casual Riders")
ax.plot(hourly_avg.index, hourly_avg["registered"], label="Registered Riders")


ax.set_title("Average Number of Riders by Hour of Day")
ax.set_xlabel("Hour of Day")
ax.set_ylabel("Average Number of Riders")
ax.legend()

plt.tight_layout()
plt.show()
```



[ ]:

Exercise 2.8

- Use the one-hot-encoded `weather` related variables to show how each weather category affects the relationships in `Exercise 2.6`.
  - **You will use the one-hot-encoded weather variables ('Cloudy', 'Storm', 'Snow') from the dataset.**
  - **Note that there are only three columns in the one-hot-encoded dataset representing Cloudy, Storm, and Snow. The fourth category (Clear) is implicit—if all three other weather columns are 0, it indicates Clear weather.**

Instructions:

- Filter the dataset based on weather conditions:
  - The dataset includes three one-hot-encoded weather variables: 'Cloudy', 'Storm', and 'Snow'.
  - Create four separate DataFrames:
    - `cloudy_df` → Rows where 'Cloudy' == 1'
    - `storm_df` → Rows where 'Storm' == 1'
    - `snow_df` → Rows where 'Snow' == 1'
    - `clear_df` → Rows where all three weather variables ('Cloudy', 'Storm', 'Snow') are 0.
- Select columns for analysis:
  - Use the same set of variables from Exercise 2.6 for visualization:

```
cor_columns = [
    'hour', 'holiday', 'temp', 'atemp', 'workingday', 'hum', 'windspeed',
    'counts', 'casual', 'registered', 'fall', 'summer', 'spring'
]
```

- Create scatter matrix plots for each weather category:
  - Create a dictionary of all dataframes as below :

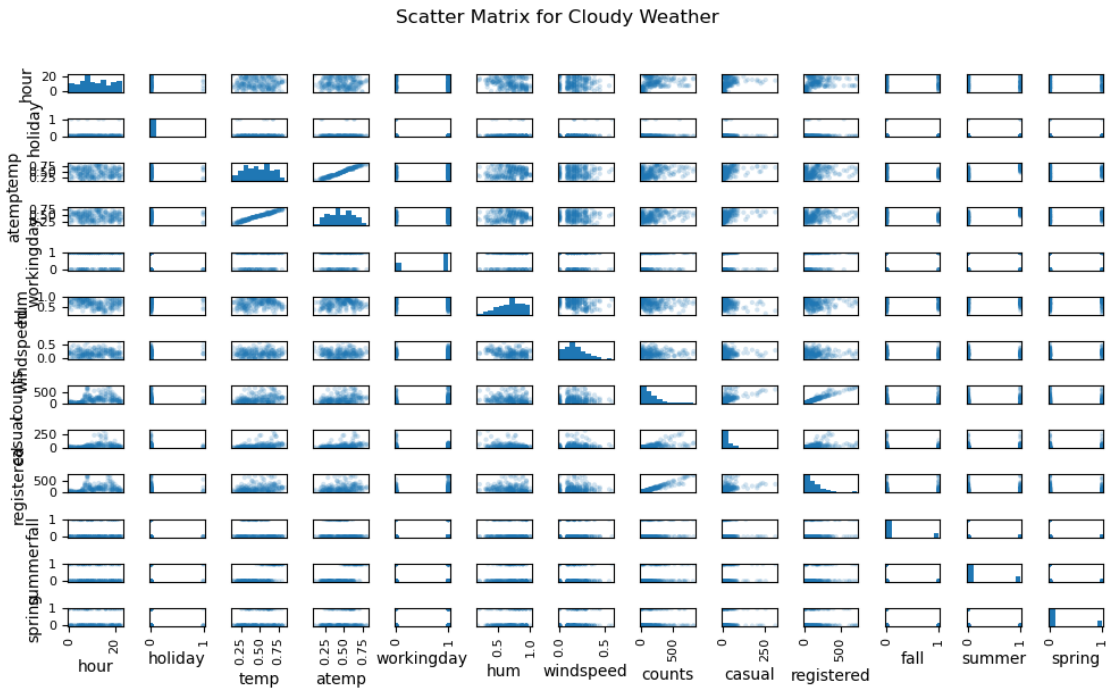
```
weather_dataframes = {
    'Cloudy': cloudy_df,
    'Storm': storm_df,
    'Snow': snow_df,
    'Clear': clear_df
}
```

- Using above dictionary plot scatter_matrix() from pandas to visualize relationships between the selected variables.
- To improve readability and performance, sample 10% of each DataFrame (if it has more than 10 rows).
- Use 'alpha=0.2' and 'diagonal="hist"' to configure the scatter matrix appearance.
- Each weather category should have its own scatter matrix plot.
  - **Hint**: Use a `for` loop iterate through `weather_dataframes` dictionary and plot the figures.

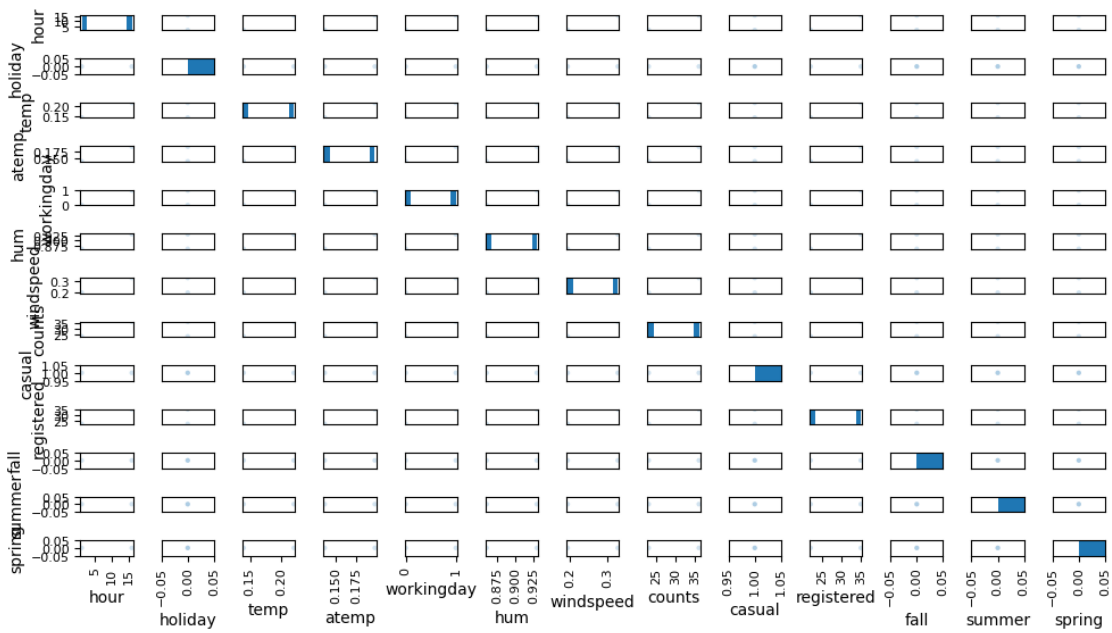
Output: - Four scatter matrix plots—one for each weather type (Cloudy, Storm, Snow, and Clear), even though there are only three columns related to weather after one-hot-encoding.

[36]:
```
"""Write your code for Exercise-2.8 here:"""

# your code here
cloudy_df = BSS_train[BSS_train["Cloudy"] == 1]
storm_df  = BSS_train[BSS_train["Storm"] == 1]
snow_df   = BSS_train[BSS_train["Snow"] == 1]

clear_df = BSS_train[
    (BSS_train['Cloudy'] == 0) &
    (BSS_train['Storm'] == 0) &
    (BSS_train['Snow'] == 0)
]

weather_dataframes = {
    "Cloudy": cloudy_df,
    "Storm": storm_df,
    "Snow": snow_df,
    "Clear": clear_df
}

cor_columns = [
    'hour', 'holiday', 'temp', 'atemp', 'workingday', 'hum', 'windspeed',
```

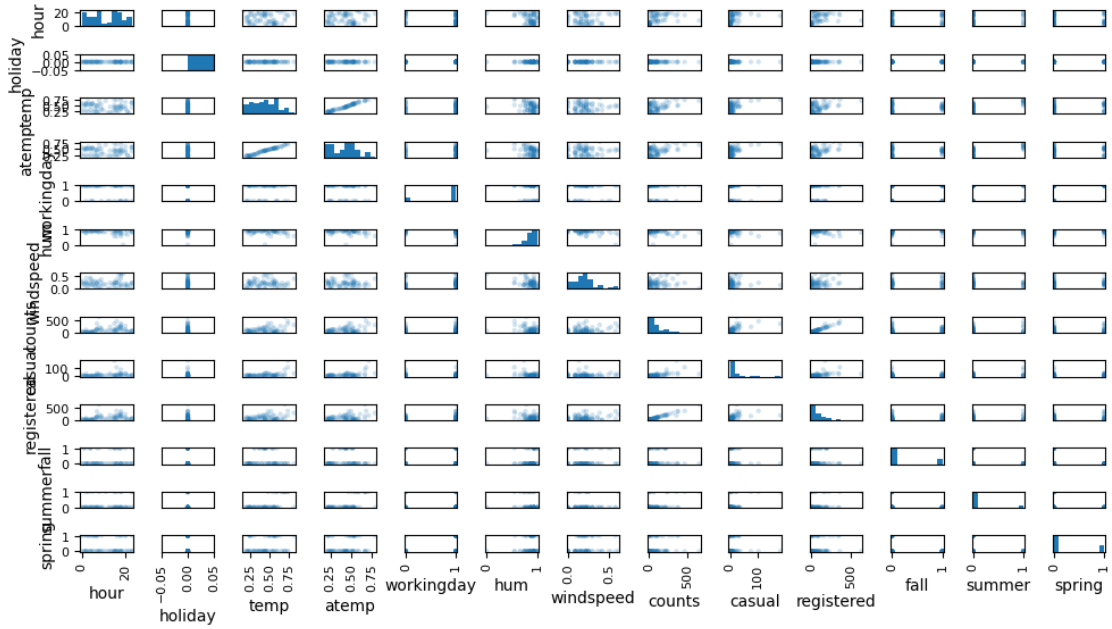
```
    'counts', 'casual', 'registered', 'fall', 'summer', 'spring'
]


for weather, df in weather_dataframes.items():


    if len(df) > 10:
        plot_df = df.sample(frac=0.1, random_state=42)
    else:
        plot_df = df

    fig, ax = plt.subplots(figsize=(10, 6))

    scatter_matrix(
        plot_df[cor_columns],
        alpha=0.2,
        diagonal="hist",
        ax=ax
    )

    plt.suptitle(f"Scatter Matrix for {weather} Weather", y=1.02)
    plt.tight_layout()
    plt.show()
```

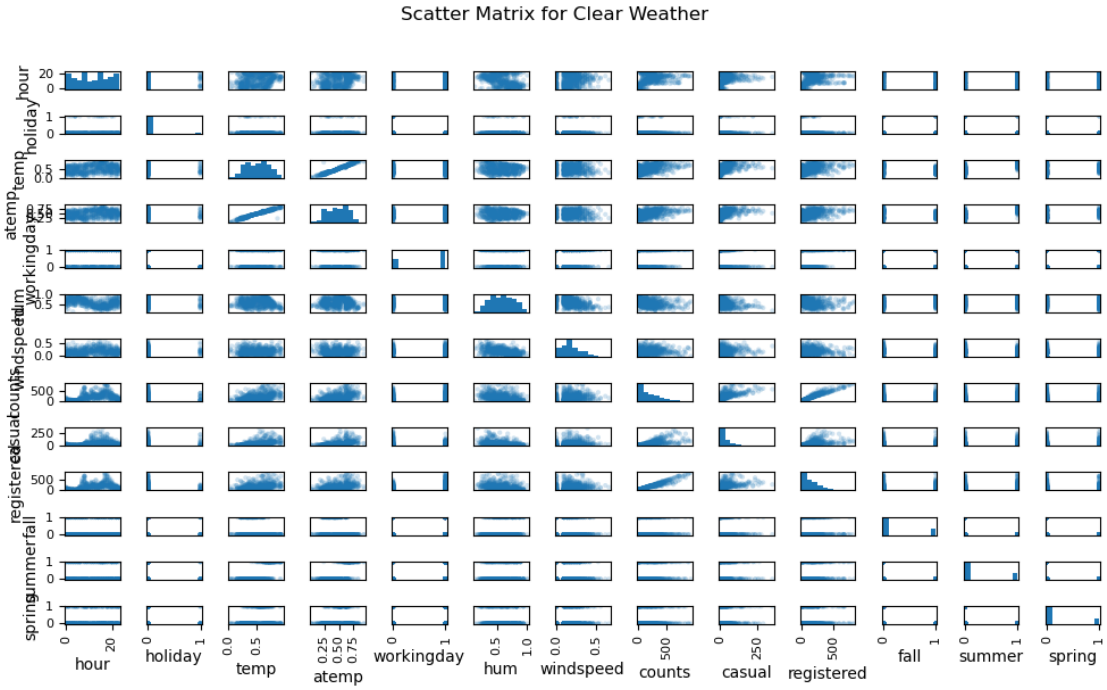


Scatter Matrix for Storm Weather



Scatter Matrix for Snow Weather




Scatter Matrix for Clear Weather
hour
holiday
temp
atemp
workingday
hum
windspeed
counts
casual
registered
fall
summer
spring

[ ]:

2 END